

Corpus analysis and text preprocessing

Corpus

- collection of machine-readable texts produced in a natural communicative setting (authentic)
- assembled with a purpose
- resampled to be representative (a sample that represents a language well) and balanced sample of a language
- different from random collections of texts

Corpus types

- general
 - e.g., The British National Corpus is supposed to represent modern British English as a whole
- specialized
 - domain (e.g., medicine or law) or genre-specific (e.g., newspaper text or academic prose)

Collecting documents

- web pages retrieved using a search engine or a web crawler
- e-mails automatically collected on e-mail servers
- customer reviews stored in a database
- newspaper articles on news servers
- scientific papers in scientific or citation databases
- posts from Facebook or Twitter discussing a common topic/originating in a region/originating in a given time span...
- responses collected during a marketing research project
- and many other sources

Available corpora

- for research and development purposes, many publicly available collections (corpora) are available
 - Reuters Corpus Volume 2 (RCV2) – almost 500,000 news stories in 13 languages
 - Text Retrieval Conference (TREC) – data sets used in past competitions
 - The British National Corpus – 100 million word collection of English written and spoken texts (with part-of-speech tagging and other properties)
 - Google N-Grams Corpus – largest English corpus at 155 billion words
 - and many others

Available corpora

- often focused on a specific domain:
 - conversations
 - multilingual corpora for translations
 - tasks (question answering, sentiment analysis, parsing, NER, ...)
 - etc.

Processing documents

- algorithms that work with document properties (features) that are related to the document content or other properties are used
- it is, therefore, necessary to define the features
- features might be extracted or derived

Document features

- characters
 - can include the full set of all characters or some filtered subset
 - are often of very limited utility; on the other hand, it is the most complete representation of a real-world text document
- sub-word level
 - some positional information (e.g., bigrams, trigrams, or word pieces that are shared across multiple words) are more useful and common
 - useful for handling out-of-vocabulary words, morphologically rich languages (contain meaningful units, like prefixes, suffixes, or stems)

Document features

- words
 - the basic level of semantics (native feature space)
 - one word = one linguistic token
 - can be combined in multi-word expressions
- terms
 - made up of specific words and expressions found within the native document
 - are generally representative

President Abraham Lincoln experienced a career that took him from log cabin to the White House.

a list of terms to represent the above document could include single-word forms such as “Lincoln,” “took,” “career,” and “cabin” as well as multiword forms like “President Abraham Lincoln,” “log cabin,” and “White House”

Document features

- concepts
 - features generated for a document by means of manual, statistical, rule-based, or hybrid categorization methodologies
 - for instance, a document collection that includes reviews of sports cars may not actually include the specific word “automotive” or the specific phrase “test drives,” but the concepts “automotive” and “test drives” might be found among the set of concepts used to identify and represent the collection
 - categorization methodologies often involve a degree of cross-referencing against an external knowledge source
 - an annotated collection of training documents
 - an existing ontology, lexicon, or formal concept hierarchy
 - a domain expert

Document features

- terms and concepts have the most expressive semantic value
- concepts require more effort to be extracted and might be domain dependent
- concepts enable better handling of synonymy (same meaning), polysemy (more meanings), hyponyms, hypernyms (sub- and superordinate meaning, e.g., *red* – *color*)

Zipf's law

- approximately valid for a natural language corpus
- states that the frequency of any word is inversely proportional to its rank in the frequency table
- the most frequent word w_1 has frequency $f(w_1)$ and rank 1, the second most frequent word w_2 has frequency $f(w_2)$ and rank 2, etc.

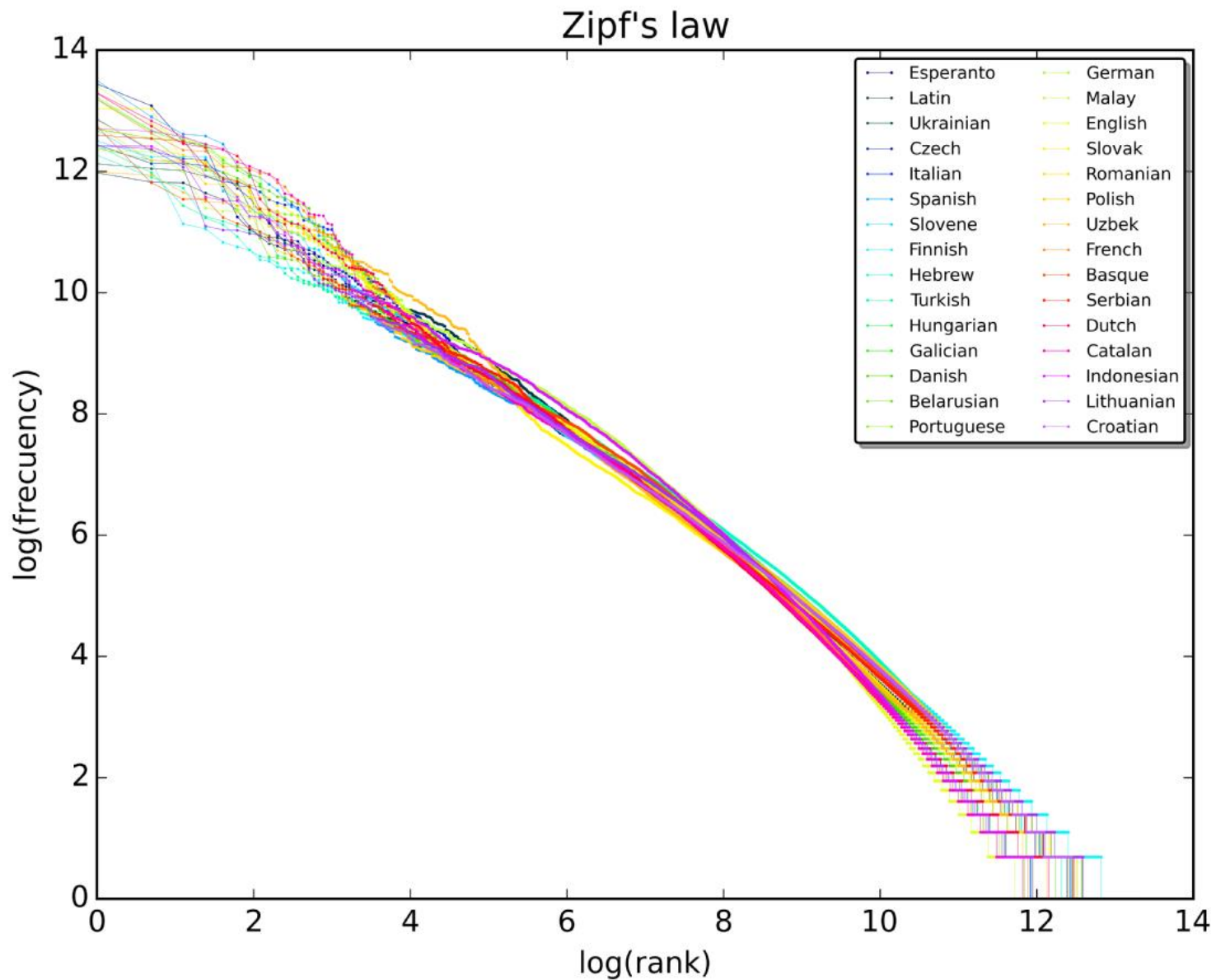
$$f(w_2) \sim f(w_1)/2, f(w_3) \sim f(w_1)/3, \dots$$

$$f(w_i) = C / \text{rank}(w_i)^a$$

- C is determined according to the corpus (ideally, $C = f(w_1)$, $a = 1$)

Zipf's law

- very often is $\log(\text{rank}(w_i))$ on the x-axis and $\log(\text{freq}(w_i))$ on the y-axis $\rightarrow$ we should see a line with slope -1
- biggest errors at the beginning and end of the graph



By SergioJimenez - Own work, CC BY-SA 4.0,
<https://commons.wikimedia.org/w/index.php?curid=45516736>

Heap's law

- an empirical law that relates the number of words in a text M to the number of unique words N
- N is approximately related to the square root of M , more accurately

$$N = K \cdot M^{\gamma}$$

N ... number of unique words

M ... total number of words

K ... a constant, typically between 10 and 100

γ ... a number between 0 and 1 (for small texts γ is close to 1, usually between 0.4 and 0.6, for a corpus with 1 mil. words (Manning, 2009) $\gamma = 0.49$)

Preprocessing purpose

- manual processing of the data is usually not feasible => automated artificial intelligence, machine learning, or statistical methods are used for solving various tasks
- these methods usually require the data in a structured format

Preprocessing purpose

- traditional text mining heavily relies on the application of various preprocessing techniques
- the goal is to infer or extract structured representations from unstructured plain textual data
- this is the most demanding activity
- modern, typically neural approaches, enable creating some representations of different pieces of texts automatically

Typical preprocessing steps

- text cleaning
- white space removal
- case folding
- spelling errors corrections
- abbreviations expanding
- stemming
- lemmatization
- stop words removal
- negation handling
- feature selection
- part-of-speech tagging
- syntactical or shallow parsing
- ...

Documents preparation

- document standardization
 - converting the document into a suitable format, e.g., extracting text from PDF documents, removing tags from HTML and XML documents, removing headers from e-mails, selecting a column from a CSV file, etc.
- determining the language of documents
- one document might be further split, e.g., a book into chapters, a newspaper article into paragraphs, a review into sentences

Linguistic processing

- tokenizing – deciding what constitutes a term
- normalizing – converting superficially different strings of characters to the same form (e.g., *car*, *Car*, *cars*, and *Cars* could all be normalized to *car*)
- annotating – marking identical strings of characters as being different (e.g., *fly* might be a verb -> *fly/VB* or a noun -> *fly/NN*)

Tokenization

- might seem simple at the first glance (e.g., for English) – words are separated by spaces (unlike, e.g., in Chinese)
- a good tokenizer must handle punctuation (e.g., *don't*, *Jane's*, *and/or*), hyphenation (e.g., *state-of-the-art* vs. *state of the art*), and recognize multi-word terms (e.g., *Barack Obama* and *ice hockey*)
- ignoring stop words – high-frequency words with relatively low information content (e.g., *the*, *of*, *and*, *them*, *who*, *that*)
- the text might be broken into character unigrams or bigrams
- the input text might be matched against a lexicon
- accurate tokenization is a challenging task for most human languages

Normalization

- based on the idea that many different strings convey the same meaning
- usually increases recall, reduces precision
- case folding
 - easy in English, can be problematic in some languages, e.g., in French, accents are optional for uppercase: example, PECHE -> pêche (fishing or peach) or péché (sin)
 - problems even in English, e.g., SMART (abbreviation of the name of an information retrieval system) vs. smart; Bush (a surname) vs. bush (a kind of plant)

Normalization

- stemming
 - the process of reducing inflected words to their stems
 - in English, affixes are simpler and more regular than in many other languages -> stemming algorithms based on heuristics work relatively well
 - Porter stemming: ED -> " (plastered -> plaster), ATIONAL -> ATE (relational -> relate), ATOR -> ATE (operator -> operate), EMENT -> " (replacement -> replac) etc.
- lemmatization
 - requires more detailed morphological analysis to convert a word into so-called lemma (dictionary form), like 'better' -> 'good' (will be missed by a stemmer)

Effects – an example

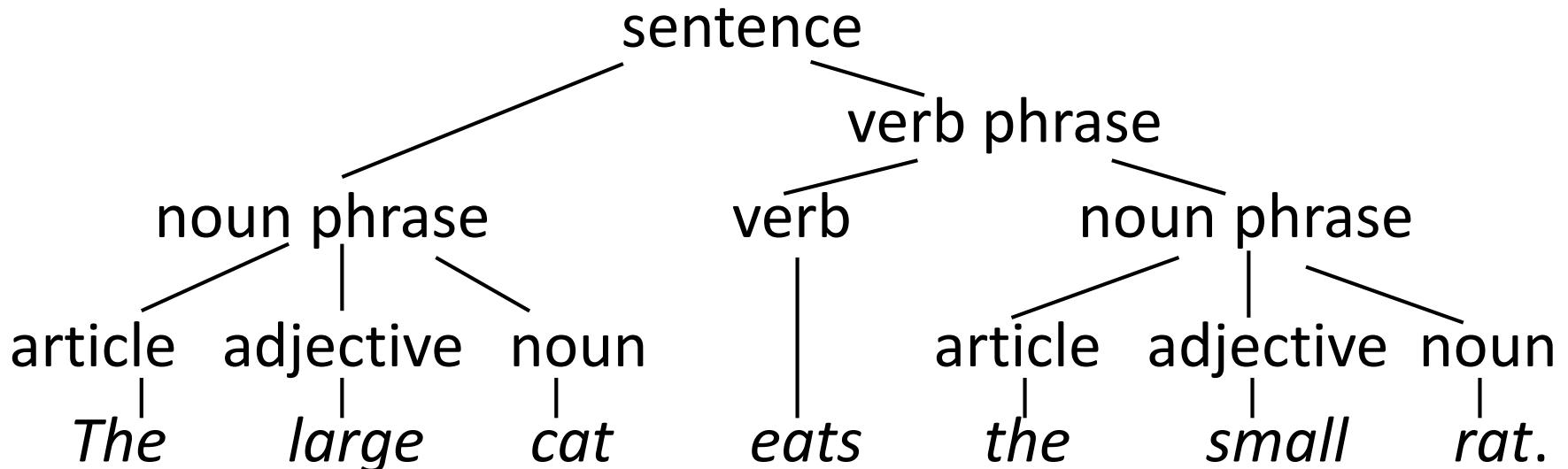
- effects of preprocessing to Reuters-RCV1 corpus (Manning et al., 2009)
 - stemming and case folding reduced the number of distinct terms by 17% each
 - the 30 most common words account for about 30% of the tokens in written text
- not only dictionary size must be considered but also the total number of terms

Annotation

- inverse to normalization
- usually increases precision and decreases recall
- part-of-speech (PoS, POS) tagging
 - marking words according to their parts of speech, i.e., determining whether a word is a noun, verb, adjective, adverb, pronoun, preposition, conjunction, interjection, numeral, or article
- word sense tagging (disambiguation)
 - some words have more meanings, e.g., *derecho* (in Spanish) is *right* or *law*
 - marking ambiguous words according to their intended meanings

Annotation

- parsing (syntax analysis)
 - analyzing the grammatical structure of sentences and marking the words in the sentences according to their grammatical roles
 - relations of a word to others and the functions of the words (subject, object, etc.) become obvious



Annotation

- shallow parsing
 - standard parsing algorithms are too expensive for use on very large corpora and are not robust enough
 - shallow parsing produces only parts that are easy and unambiguous, typically, small and simple noun and verb phrases
 - for the purposes of information extraction, shallow parsing is usually sufficient

Converting texts into a suitable format

- a document is typically broken down into some pieces (typically the words) that are not as complex as the entire document is
- these pieces might be then transformed in some way (relevant features are derived):
 - removed (e.g., rare words are eliminated)
 - changed to a different value (e.g., stemmed)
 - supplemented by additional information (e.g., part of speech, derived concepts)
- the features are represented in a way suitable for a particular algorithm (e.g., feature–document matrix)

Converting texts into a suitable format

- people for order the words usually of important the is

Converting texts into a suitable format

- people for order the words usually of important the is
(for people, the order of words is usually important)
- however, the word order is additional information => increased complexity
- Is it really necessary to deal with word order?

Converting texts into a suitable format

- combination vs. permutation – example
 - 50 unique words
 - a document consisting of 1000 words (repetitions allowed)
 - number of combinations (order doesn't matter):
5.49e84
 - number of permutations (order does matter):
9.33e1698
- The question is: How much does the word order matter?

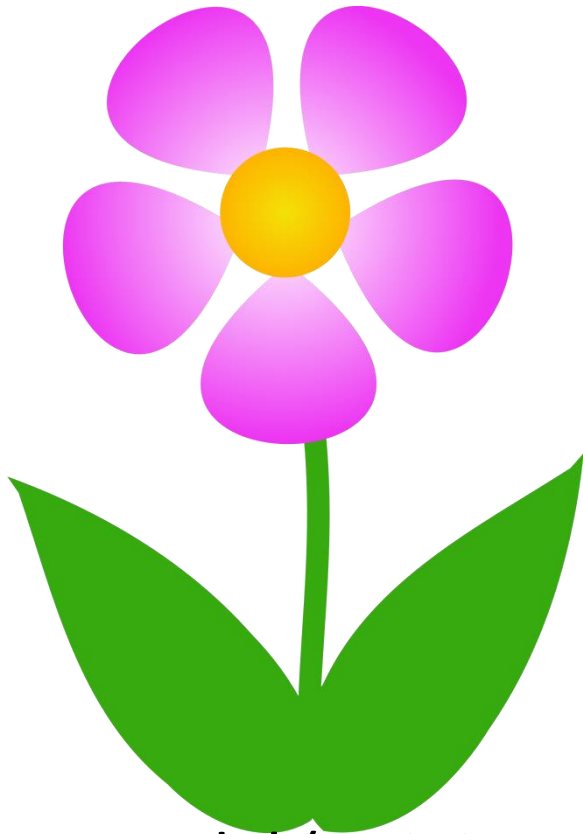
An example – image classification

Is this a car or a flower?



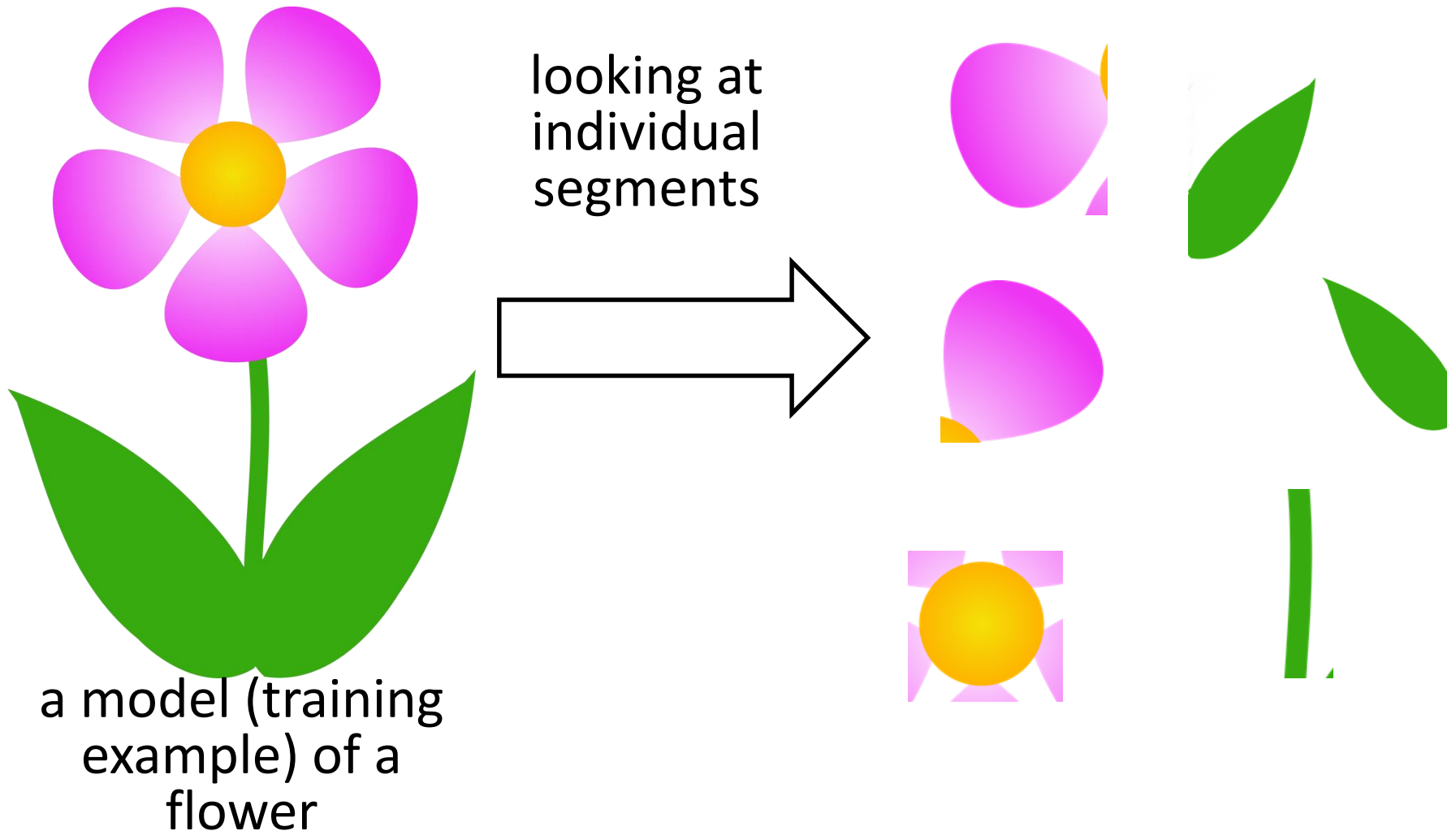
What might be needed to answer this question?

An example – image classification



a model (training
example) of a
flower

An example – image classification



An example – image classification



a model (training
example) of a car

An example – image classification

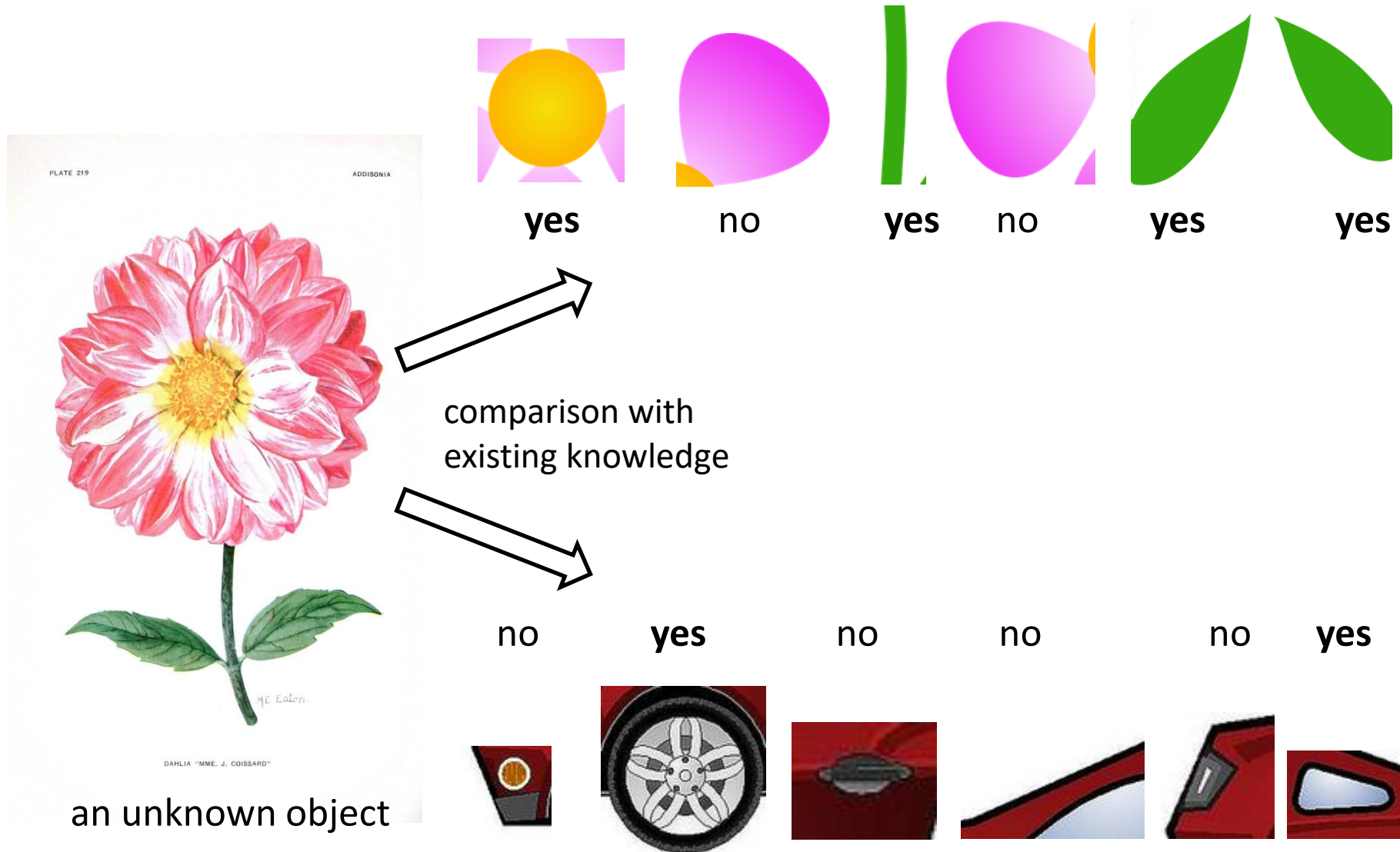


a model (training example) of a car

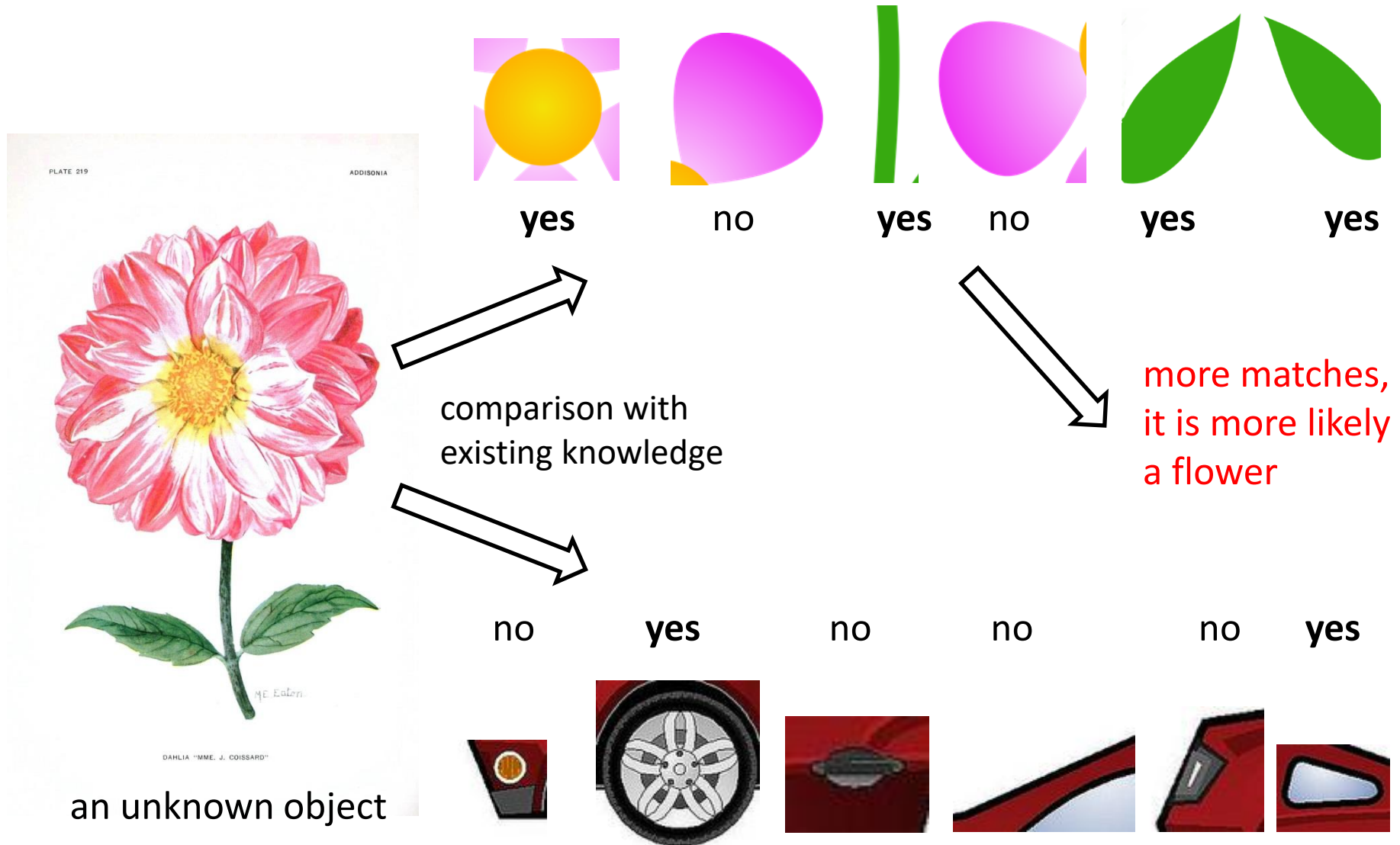
looking at individual segments



An example – image classification



An example – image classification



Converting texts into a suitable format

- documents are often treated as a bag-of-words
- bag = multiset – a set where duplicates are allowed
- not important are
 - the order of words
 - relationships between the words
- the problem becomes less complex and thus manageable

Document vector representation

- a form suitable for data mining algorithms (tabular format)
- a document is represented by a vector
 - attributes – numeric values representing the properties of the document
 - properties – terms (included in the text), concepts (derived from the text)
 - attribute values – characterize and quantify the features

Document vector representation – a simple example

- a set of documents (reviews)

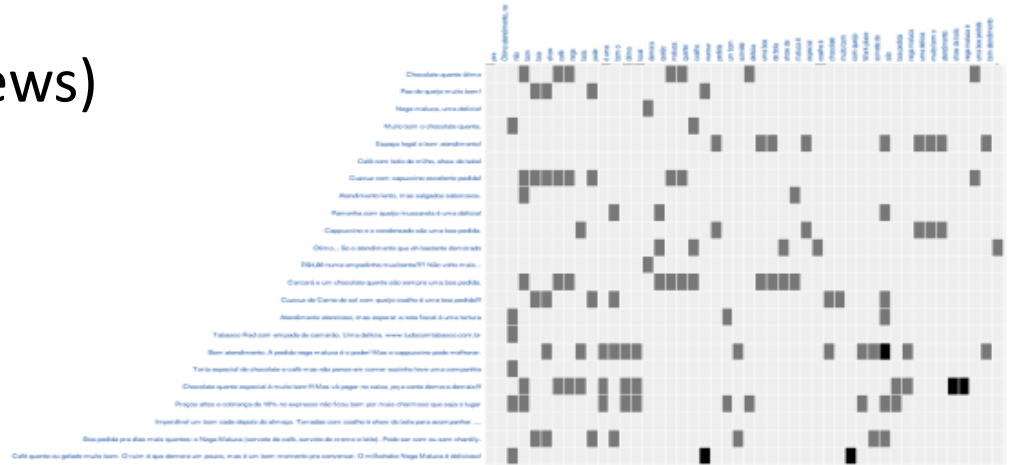
very good product

very bad product

very very good

good

quite bad



- a simple document vector representation

good	quite	product	bad	very
1	0	1	0	1
0	0	1	1	1
1	0	0	0	2
1	0	0	0	0
0	1	0	1	0

Vector components

- a *local weight* L_{ij} representing the frequency f_{ij} of term i in document j (in every single document)
- a *global weight* G_i reflecting the discriminative ability of the term i , based on the distribution of the term over the entire document collection
- a *normalization factor* N_j correcting the impact of different lengths of documents

$$w_{ij} = L_{ij} * G_i / N_j$$

w_{ij} ... the weight of i -th term in j -th document

i ... the number of a term

j ... the number of a document

- detailed review: Dařena, F. VecText: Converting Documents to Vectors. *IAENG International Journal of Computer Science*, 2019, 46(2): 170-177.

Local weights

- term presence (binary)

$$f_{ij} = 0 \Rightarrow w_{ij} = 0, \quad f_{ij} > 0 \Rightarrow w_{ij} = 1$$

- term frequency

$$w_{ij} = f_{ij}$$

- logarithm

– de-emphasize the effect of frequency (a term appearing 10 times is not 10 times more important than a term appearing once)

$$f_{ij} = 0 \Rightarrow w_{ij} = 0, \quad f_{ij} > 0 \Rightarrow w_{ij} = 1 + \log f_{ij}$$

Local weights

- augmented normalized term frequency
 - assigns weight k (typically 0.5) to every term in a document + a bonus up to $1-k$
 - for longer documents lower (e.g., 0.3), for shorter documents higher (e.g., 0.5)
$$f_{ij} = 0 \Rightarrow w_{ij} = 0, \quad f_{ij} > 0 \Rightarrow w_{ij} = k + (1-k) * f_{ij} / x_j$$

(x_j is the maximum frequency in document j)
- and many others

Global weights

- inverse document frequency (the most popular)

$$idf(t_i) = \log \frac{N}{n(t_i)}$$

N ... number of documents, $n(t_i)$... number of documents containing term t_i (document frequency)

- other variations, e.g., squared inverse document frequency, probabilistic inverse document frequency, GFIDF, Entropy, ...

Normalization

- transforms the elements of vector d_j , i.e., the vector with local and global weighting applied:

$$d_j = (l_{1j} * g_1, l_{2j} * g_2, \dots, l_{mj} * g_m)$$

- eliminates the problem of preferring longer documents because
 - they have higher term frequencies
 - contain more words (e.g., the probability of matching against a query is higher)

Normalization

- cosine normalization (L2-norm, magnitude, length)
 - the most popular

$$n_j = \sqrt{\sum_{i=1}^m d_i^2}$$

- other variants – max weight, sum of weights, ...

Formats – ARFF (Attribute-Relation File Format)

- developed for Weka
- header section
 - @relation <relation-name>
 - @attribute <attribute-name> <datatype>
 - data type – numeric, string, nominal specification (e.g., @attribute class {negative, positive, neutral}, date (with optional format, e.g., @attribute timestamp date "yyyy-MM-dd HH:mm:ss")
- data section
 - @data
 - comma separated list of values
 - in the order specified in the header
 - strings are quoted, missing values represented by ?

Formats – ARFF example

@RELATION iris

@ATTRIBUTE sepallength NUMERIC

@ATTRIBUTE sepalwidth NUMERIC

@ATTRIBUTE petallength NUMERIC

@ATTRIBUTE petalwidth NUMERIC

@ATTRIBUTE class {Iris-setosa,Iris-versicolor,Iris-virginica}

@DATA

5.1,3.5,1.4,0.2,Iris-setosa

4.9,3.0,1.4,0.2,Iris-setosa

4.7,3.2,1.3,0.2,Iris-setosa

4.6,3.1,1.5,0.2,Iris-setosa

5.0,3.6,1.4,0.2,Iris-setosa

...

Formats – Sparse ARFF

- zero values (not missing!) are omitted
- header section
 - the same as ARFF
- data section

@data

0, X, 0, Y, "class A" →

0, 0, W, 0, "class B"

– indices counted from 0

@data

{1 X, 3 Y, 4 "class A"}

{2 W, 4 "class B"}

Formats – C5

- several files where extensions are added to a filestem
 - names – description of the application's attributes (required)
 - data – cases used to generate a classifier (required)
 - test – unseen cases used to test a classifier (optional)
 - cases – cases to be classified subsequently (optional)
 - costs – differential misclassification costs (optional)
 - tree – decision tree classifier produced by C5.0 (output)
 - rules – ruleset classifier produced by C5.0 (output)

Formats – C5

- classes
 - a list of class names separated by commas, e.g., *positive, negative, neutral*
 - the name of a discrete attribute (the target attribute) that contains the class value, e.g., *sentiment*
 - the name of a continuous attribute followed by a colon and one or more thresholds in increasing order and separated by commas (e.g., *age: 12, 19* would define three classes: *age* $\leq$ 12, 12 < *age* $\leq$ 19, and *age* > 19)

Formats – C5

- attributes
 - *attribute name: type*
 - types
 - continuous – numeric values
 - date – dates in the form YYYY/MM/DD or YYYY-MM-DD
 - time – times in the form HH:MM:SS
 - timestamp – times in the form YYYY/MM/DD HH:MM:SS or YYYY-MM-DD HH:MM:SS
 - a comma-separated list of names – allowable set of discrete values (e.g., *food_type: meat, poultry, fish, vegetables* or *grade: [ordered] low, medium, high*)
 - discrete N – discrete, unordered values, assembled from the data itself; *N* is the maximum number of such values (not for the target attribute)
 - ignore – the values of the attribute should be ignored
 - label – label for each case, ignored when classifiers are constructed, used when referring to individual cases

Formats – C5 example

hypothyroid.names

diagnosis. | the target attribute

age: continuous.

sex: M, F.

on thyroxine: f, t.

...

TSH: continuous.

TT4: continuous.

T4U: continuous.

FTI:= TT4 / T4U.

referral source: WEST, STMW, SVHC, SVI, SVHD, other.

diagnosis: primary, compensated, secondary, negative.

ID: label.

hypothyroid.data

41,F,f,f,f,f,f,f,f,f,f,f,f,f,f,1.3,2.5,125,1.14,SVHC,negative,3733

23,F,f,f,f,f,f,f,f,f,f,f,f,f,f,4.1,2,102,?,other,negative,1442

46,M,f,f,f,f,N/A,f,f,f,f,f,f,f,f,f,0.98,?,109,0.91,other,negative,2965

Formats – CLUTO

- sparse matrix format
 - first line: <no. of rows> <no. of columns> <no. of non-zero entries>
 - other lines: information about non-zero entries as <column number> <value>
 - column numbers start from 1, ordering is not important
- dense matrix format
 - first line: <no. of rows> <no. of columns>
 - other lines: space separated values of all entries

Formats – CLUTO

- sparse matrix format – example

	1.1			-0.5			0.2
1.4	0.4		-0.4				
		1.8			2.0		3.0
1.0							
	5.5		3.0			8.0	
		1.0		-1.0	2.0		
	3.5		-1.0	4.0		2.0	8.0



Matrix Input File

```
7  8  21
2  1.1  5  -0.5  8  0.2
1  1.4  2  0.4  4  -0.4
3  1.8  6  2.0  8  3.0
1  1.0
2  5.5  4  3.0  7  8.0
3  1.0  5  -1.0  6  2.0
2  3.5  4  -1.0  5  4.0  7  2.0  8  8.0
```

Formats – CSV

- a universal format supported by many applications
- attribute values are separated by commas, sometimes also by tabulators
- the first line might contain attribute names